```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df=pd.read_csv("/content/CustomerChurn.csv")
df
```

```
{"type":"dataframe","variable_name":"df"}
```

```
df.head()  #Shows the first 5 rows of the DataFrame (by default) Eg:-
df.head(10) → shows first 10 rows.
```

```
{"type":"dataframe","variable_name":"df"}
```

```
df.tail() #Shows the last 5 rows of the DataFrame (by default). Eg:-
df.tail(3) → shows last 3 rows.
```

```
{"type":"dataframe"}
```

```
#Gives a summary of the DataFrame including:
#Column name,
#Data types (int64, float64, object, etc.),
#Number of non-null (non-empty) values in each column,
#Memory usage

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
```

```
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
df["TotalCharges"]=df["TotalCharges"].replace(" ","0")  # replacing blank value with 0.
df["TotalCharges"]=df["TotalCharges"].astype("float")  #converting object datatype into float.
```

Replacing blanks with 0 as tenure is 0 and no total charges are recorded.

```
df.info()  #Monthly charges have now been converted from object datatype to float.
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   float64
 20  Churn             7043 non-null   object
dtypes: float64(2), int64(2), object(17)
memory usage: 1.1+ MB
```

```
df.isnull().sum().sum() #Checks for null values
```

```
np.int64(0)
```

```
df.describe()
```

{"summary":"{\n \"name\": \"df\",\n \"rows\": 8,\n \"fields\": [\n{\n   \"column\": \"SeniorCitizen\",\n   \"properties\": {\n\"dtype\": \"number\",\n   \"std\": 2489.9992387084,\n\"min\": 0.0,\n   \"max\": 7043.0,\n\"num_unique_values\": 5,\n   \"samples\": [\n0.1621468124378816,\n   1.0,\n   0.36861160561002687\n],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n}\n   },\n   {\n   \"column\": \"tenure\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 2478.9752758409018,\n   \"min\": 0.0,\n   \"max\": 7043.0,\n\"num_unique_values\": 8,\n   \"samples\": [\n32.37114865824223,\n   29.0,\n   7043.0\n   ],\n\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"MonthlyCharges\",\n\"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 2468.7047672837775,\n   \"min\": 18.25,\n   \"max\": 7043.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n64.76169246059918,\n   70.35,\n   7043.0\n   ],\n\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"TotalCharges\",\n\"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 3122.5732655623974,\n   \"min\": 0.0,\n   \"max\": 8684.8,\n\"num_unique_values\": 8,\n   \"samples\": [\n2279.7343035638223,\n   1394.55,\n   7043.0\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n}\n   }\n]\n}","type":"dataframe"}

```
df["customerID"].duplicated().sum() #checks duplicates in customerID column.
```

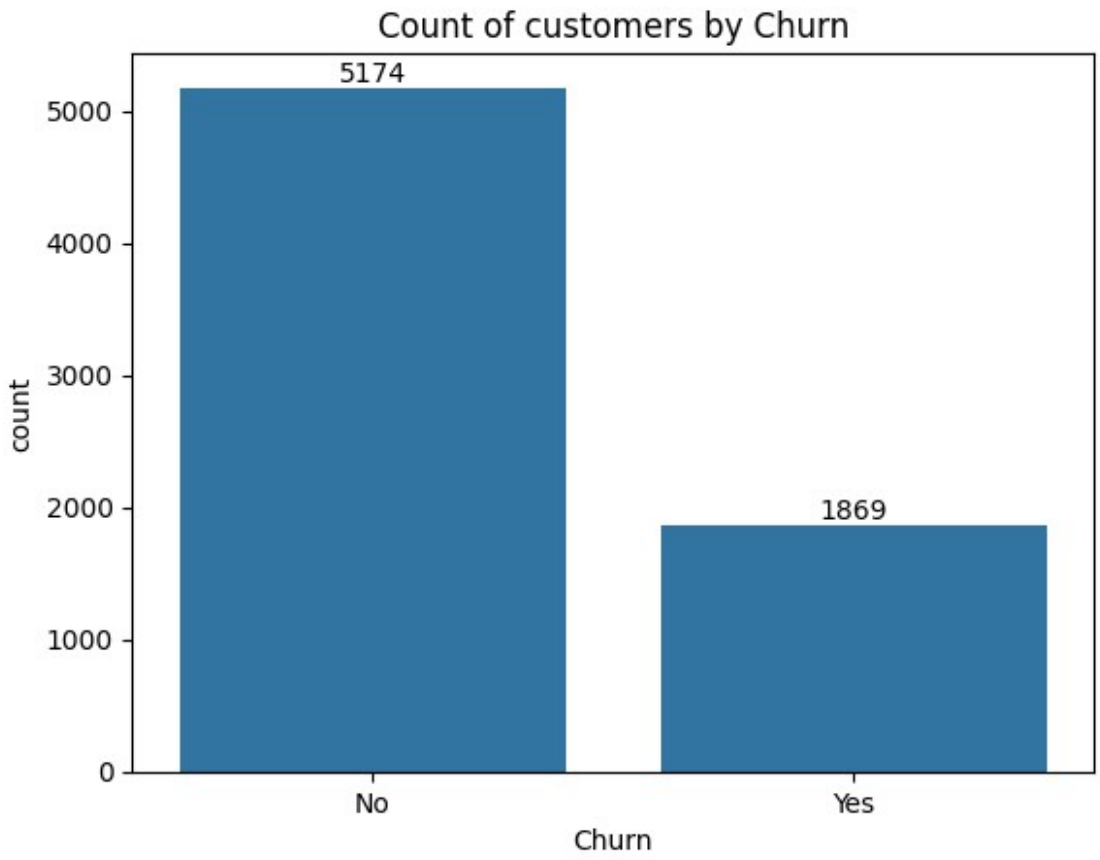
```
np.int64(0)
```

Converted 0 and 1 values of senior citizen to yes/no to make it easier to understand.

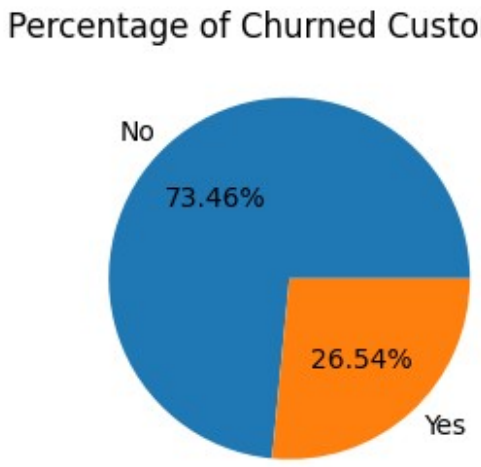
```
def conv(value):
  if value == 1:
    return "yes"
  else:
    return "no"
df['SeniorCitizen']=df["SeniorCitizen"].apply(conv)

df.head(30)
```

{"type":"dataframe","variable_name":"df"}

```
ax=sns.countplot(x='Churn',data=df)
ax.bar_label(ax.containers[0])
plt.title("Count of customers by Churn")
plt.show()
```

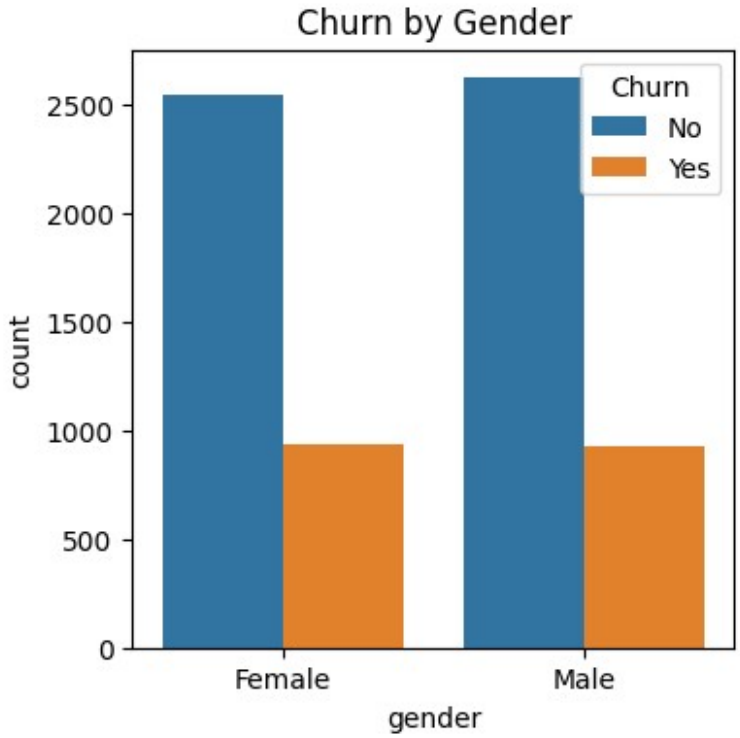



```
plt.figure(figsize=(3,4))
gb=df.groupby("Churn").agg({'Churn':"count"})
plt.pie(gb['Churn'],labels=gb.index,autopct="%1.2f%%" )
plt.title("Percentage of Churned Customers.")
plt.show()
```

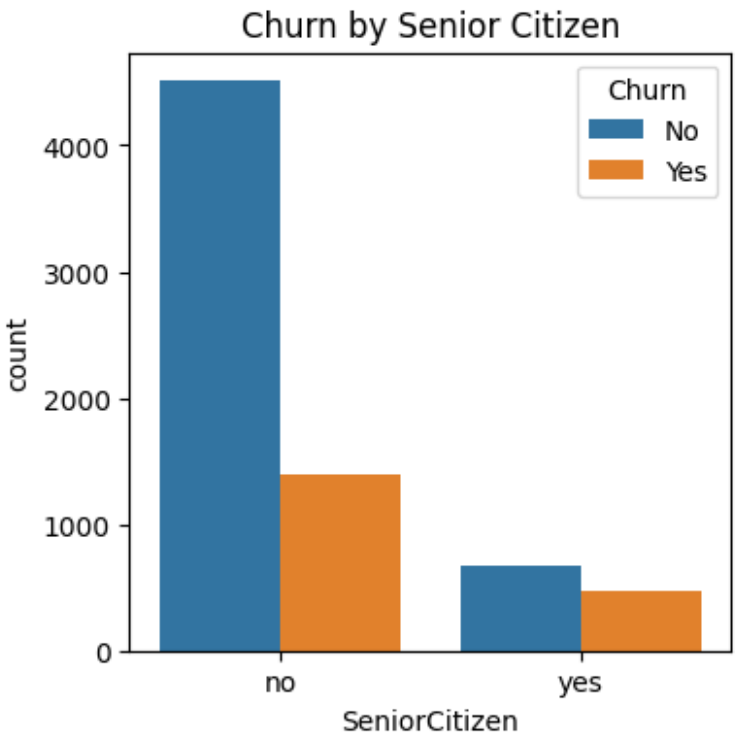


From the given pie chart we can conclude that 26.54% of our customers have churned out. Now lets explore the reason behind it.

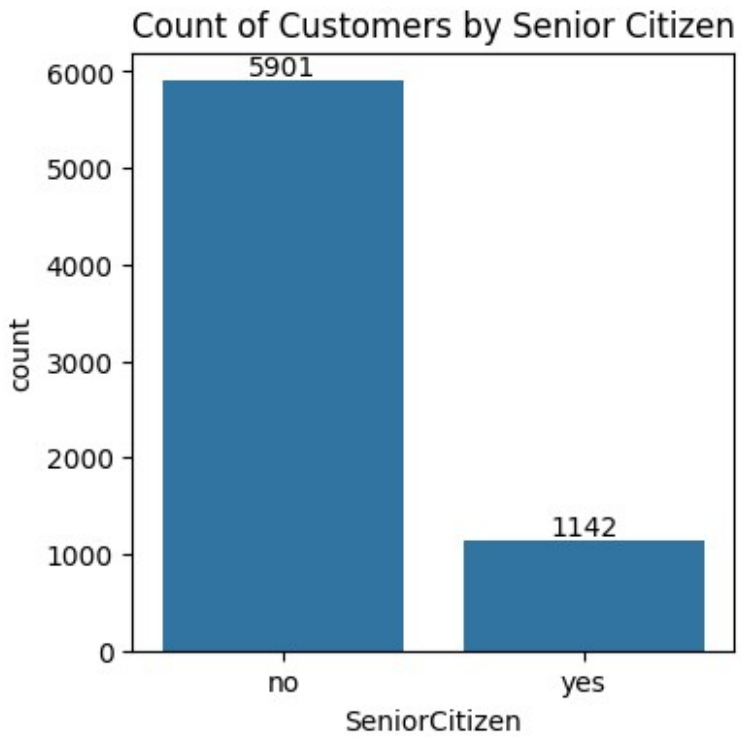
```
plt.figure(figsize=(4,4))
sns.countplot(x="gender",data=df,hue="Churn")
plt.title("Churn by Gender")
plt.show()
```




```
plt.figure(figsize=(4,4))
sns.countplot(x="SeniorCitizen",data=df,hue="Churn")
plt.title("Churn by Senior Citizen")
plt.show()
```

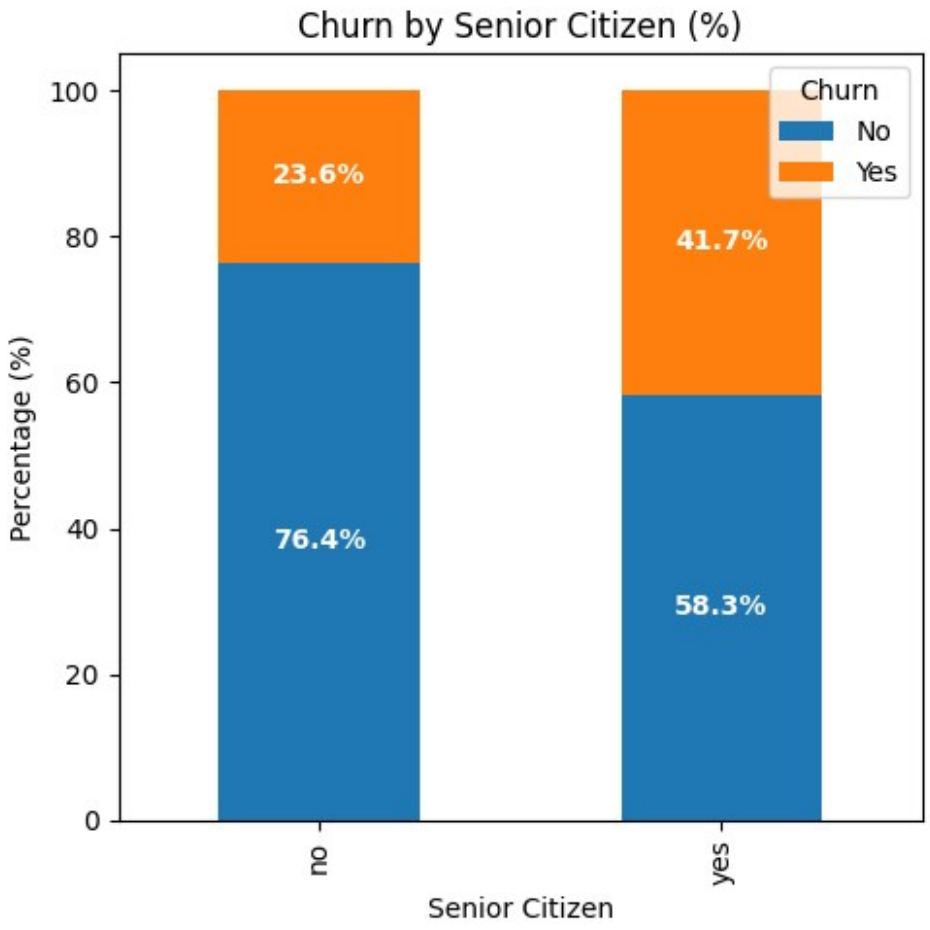



```
plt.figure(figsize=(4,4))
ax= sns.countplot(x="SeniorCitizen",data=df)
ax.bar_label(ax.containers[0])
plt.title("Count of Customers by Senior Citizen")
plt.show()
```



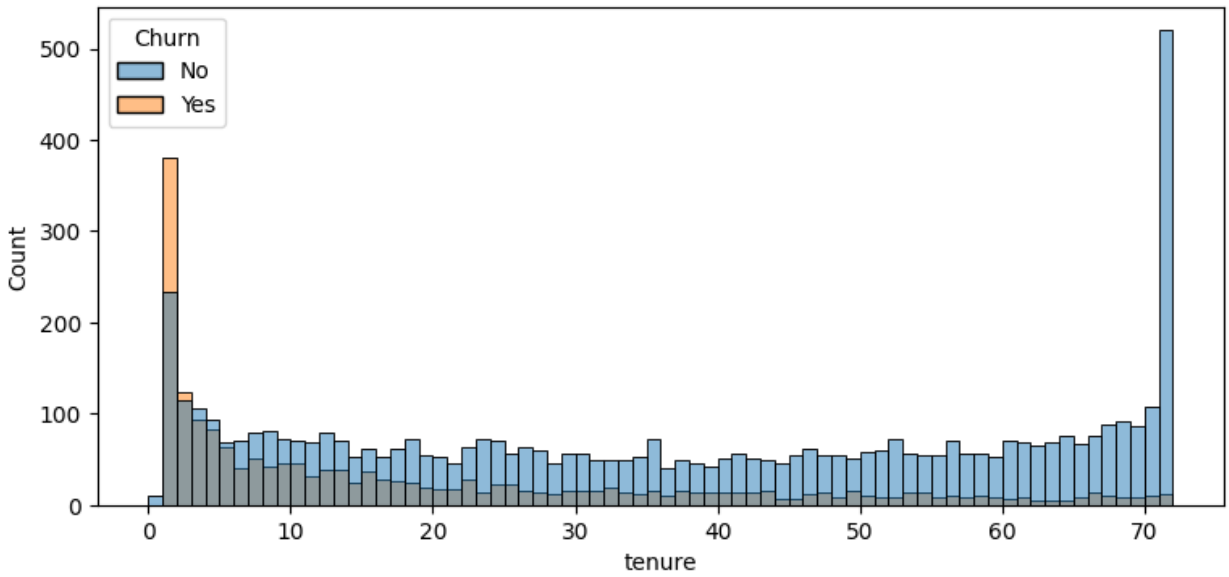

```
# Calculate counts and percentages within each SeniorCitizen group
grouped = (
    df.groupby(['SeniorCitizen', 'Churn'])
    .size()
    .reset_index(name='Count')
)

# Calculate percent within each SeniorCitizen group using transform
grouped['Percent'] = grouped.groupby('SeniorCitizen')
['Count'].transform(lambda x: 100 * x / x.sum())

# Pivot the data for plotting
pivot_df = grouped.pivot(index='SeniorCitizen', columns='Churn',
values='Percent')

# Plot stacked bar chart
pivot_df.plot(kind='bar', stacked=True, figsize=(5,5),
color=['#1f77b4', '#ff7f0e'])

# Add title and labels
plt.title("Churn by Senior Citizen (%)")
plt.ylabel("Percentage (%)")
plt.xlabel("Senior Citizen")

# Add percentage labels on bars
for i, row in enumerate(pivot_df.values):
    cum = 0
```

```
    for val in row:
        if pd.notna(val) and val > 0: # Only add labels for non-zero
percentages
            plt.text(i, cum + val/2, f"{val:.1f}%", ha='center',
va='center', color='white', fontsize=10, fontweight='bold')
        cum += val

plt.legend(title="Churn", loc='upper right')
plt.tight_layout()
plt.show()
```



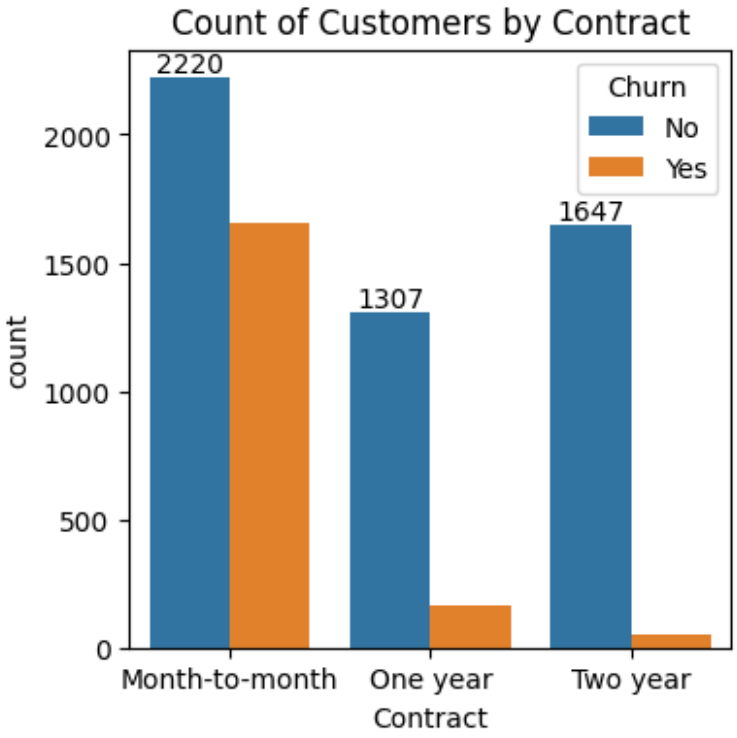
Comaparatively a greater percentage of people in Senior Citizen category have Churned out.

```
plt.figure(figsize=(9,4))
sns.histplot(x="tenure",data= df,bins= 72,hue= "Churn")
plt.show()
```



People who have used our services for a long time have stayed and people who have used our services for a 1/2 months have churned.

```
plt.figure(figsize=(4,4))
ax= sns.countplot(x="Contract",data=df,hue="Churn")
ax.bar_label(ax.containers[0])
plt.title("Count of Customers by Contract")
plt.show()
```

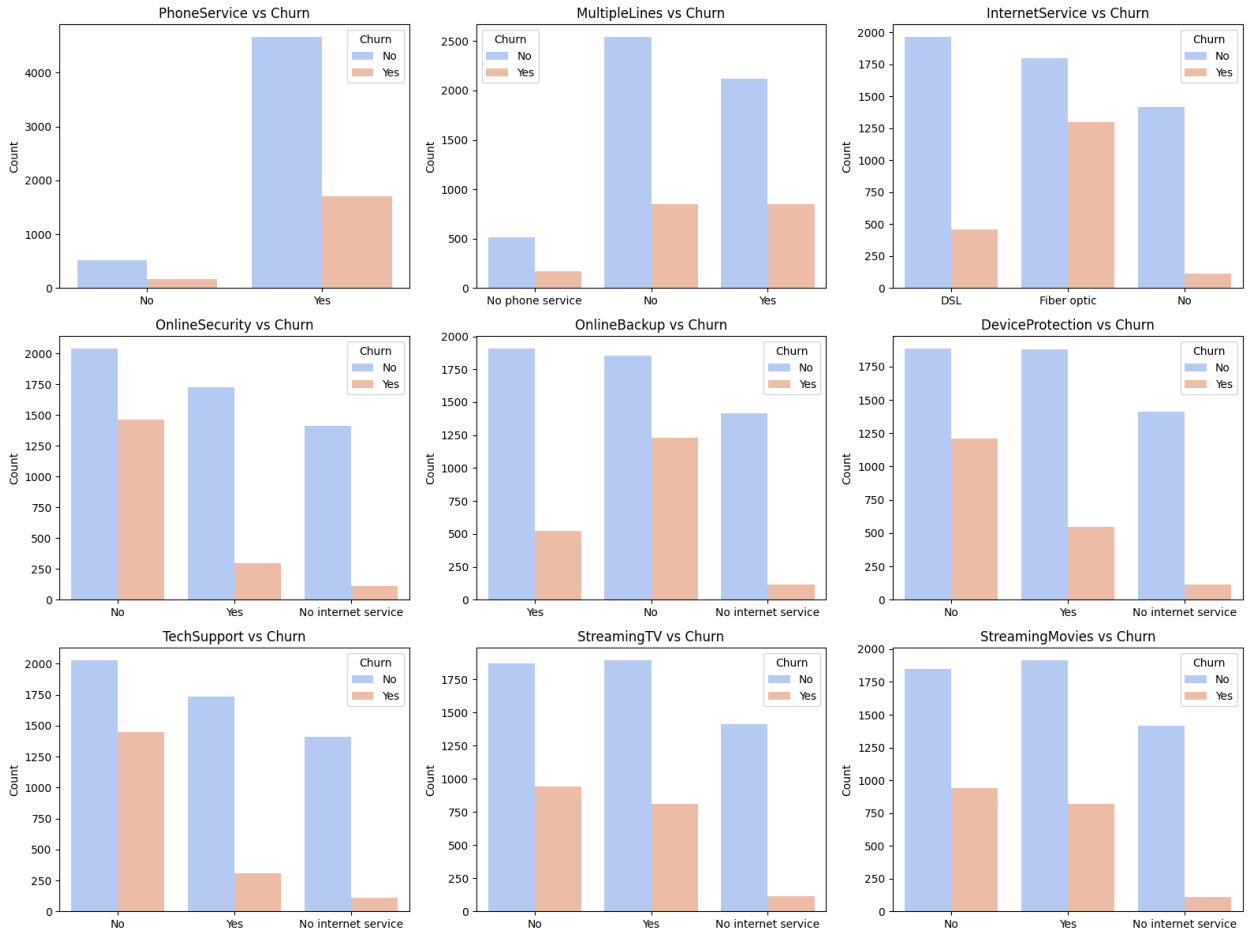


People who have month-to-month contract are likely to churn than from those who have 1 or 2 years of contract.

```
df.columns.values
```

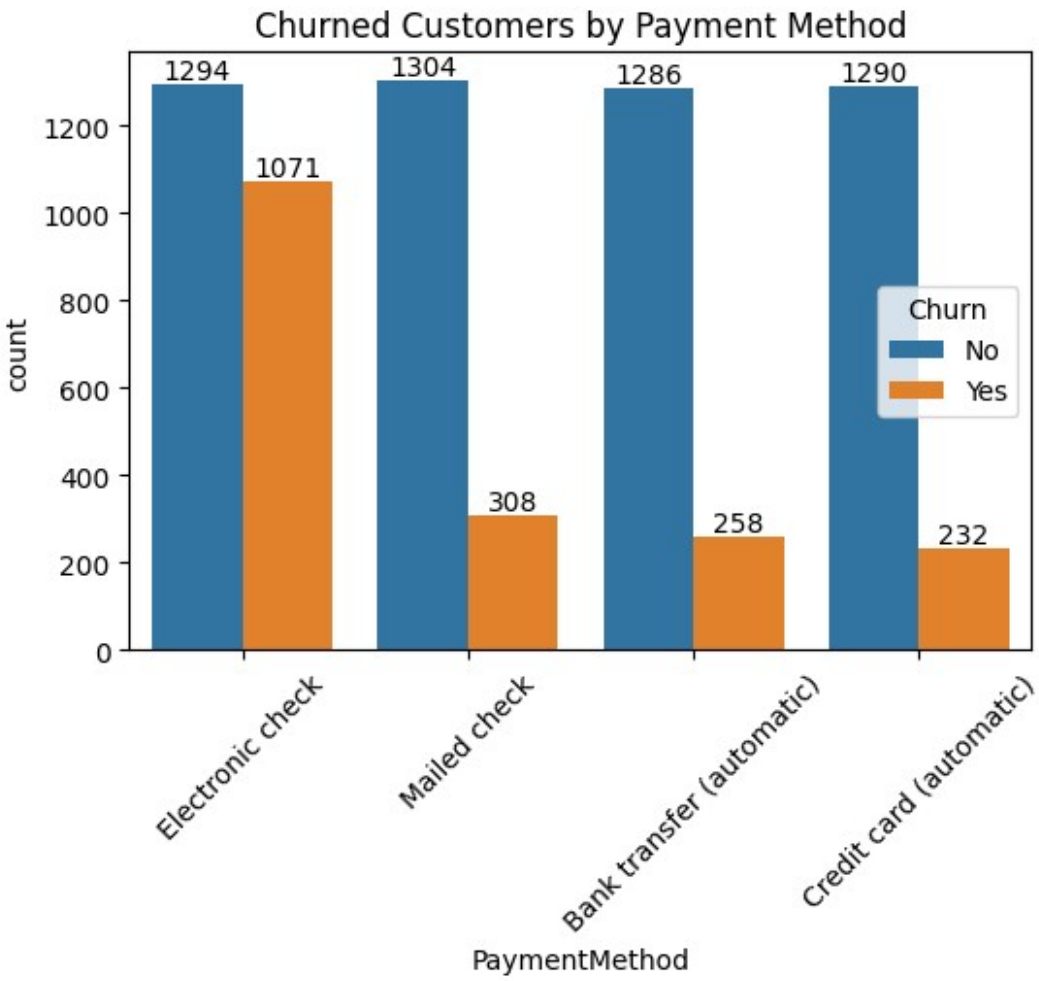
```
array(['customerID', 'gender', 'SeniorCitizen', 'Partner',
'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
       'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract',
       'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges',
       'TotalCharges', 'Churn'], dtype=object)
```

```
# List of columns
cols = [
    'PhoneService', 'MultipleLines', 'InternetService',
    'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
    'TechSupport', 'StreamingTV', 'StreamingMovies'
]

# Set figure size (adjust width & height as needed)
plt.figure(figsize=(16, 12))

# Loop through each column and create subplots
for i, col in enumerate(cols, 1):
    plt.subplot(3, 3, i)   # 3 rows × 3 columns grid
    sns.countplot(x=col, data=df, hue="Churn", palette="coolwarm")
    plt.title(f"{col} vs Churn", fontsize=12)
    plt.xlabel("")   # Remove x-labels to keep clean
    plt.ylabel("Count")

plt.tight_layout()
plt.show()
```



The majority of customers who do not churntend to have services like PhoneServices,InternetServices(particularly DSL), and OnlineSecurity enabled. For services like OnlineBackup, TechSupport, and StreamingTV, churn rates are noticeably higher when these services are not used or are unavailable.

```
plt.figure(figsize=(6,4))
ax= sns.countplot(x="PaymentMethod",data=df,hue="Churn")
ax.bar_label(ax.containers[0])
ax.bar_label(ax.containers[1])
plt.title("Churned Customers by Payment Method")
plt.xticks(rotation = 45)
plt.show()
```



Customer is likely to churn when they are using electronic check as a Payment Method.